COG 403 Final Project

Do Language Model Surprisal Scores Predict Human Reading Times?

Comparing GPT-2 and BERT on the Natural Stories Corpus

Course: COG 403 – Computational Psycholinguistics
Dataset: Natural Stories Corpus (Futrell et al., 2018)
Models: GPT-2 (autoregressive LM) · BERT (masked LM)

Overview

A central question in computational psycholinguistics is whether the **surprisal** of a word under a language model — how unexpected the model finds it given prior context — predicts how long human readers spend on that word.

This project tests that hypothesis using two modern neural language models:

Model	Architecture	Surprisal type
GPT-2	Causal / autoregressive transformer	$-\log P(w_n \mid w_1 \ldots w_{n-1})$
BERT	Bidirectional masked transformer	$-\log P(w_n \mid \text{all other words})$

The Natural Stories Corpus provides self-paced reading times for 10 naturalistic English stories (~10 000 unique word tokens, ~1 million RT observations across participants).

Key questions:

1. Do higher model surprisal values correlate with slower reading times?
2. Which model is a better predictor of human reading behaviour?
3. How do the two models' surprisal estimates compare to each other?

1. Imports & Setup

In [1]:
```
import os
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
import statsmodels.formula.api as smf

# ── Plotting style ──────────────────────────────────────────────────────
plt.rcParams.update({
    "figure.dpi": 130,
    "figure.facecolor": "white",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "DejaVu Sans",
    "axes.titlesize": 13,
    "axes.labelsize": 11,
})
COLORS = {"gpt2": "#4C72B0", "bert": "#DD8452", "neutral": "#55A868"}

print("All imports successful ✓")
```

```
All imports successful ✓
```

2. Load Pre-computed Data

Surprisal values were pre-computed on GPU (CUDA) using:

- **GPT-2** (`gpt2` , 117M params) via chunked sliding-window inference (window=1024, stride=512)
- **BERT** (`bert-base-uncased` , 110M params) via whole-word masked prediction

Results are loaded directly from the saved CSV — no GPU needed.

In [2]:
```
# ── Load surprisals CSV ─────────────────────────────────────────────────
CSV_PATH = "df_final_with_surprisals.csv"    # ← place your CSV here

df = pd.read_csv(CSV_PATH)
print(f"Loaded {len(df):,} rows  |  {df['subject'].nunique()} participants  |  {df[
print()
print(df[["token", "story_id", "position", "rt", "gpt2_surprisal", "bert_surprisal"
```

```
Loaded 1,013,377 rows  |  181 participants  |  10 stories

  token  story_id  position    rt  gpt2_surprisal  bert_surprisal
0    If         1         1  3061        3.259766        0.004272
1    If         1         1  1527        3.259766        0.004272
2    If         1         1  1272        3.259766        0.004272
3    If         1         1  1002        3.259766        0.004272
4    If         1         1  1404        3.259766        0.004272
5    If         1         1  4750        3.259766        0.004272
6    If         1         1  1027        3.259766        0.004272
7    If         1         1  2120        3.259766        0.004272
8    If         1         1   605        3.259766        0.004272
9    If         1         1  2370        3.259766        0.004272
```

3. Data Cleaning & Preprocessing

Standard preprocessing for self-paced reading studies:

- Drop rows with missing surprisal values (boundary words lost during chunking)
- Remove implausibly fast (< 100 ms) or slow (> 3000 ms) RTs — likely button-press errors or distraction
- Log-transform RTs to reduce positive skew (standard in the psycholinguistics literature)

In [3]:
```
# ── Drop missing ──────────────────────────────────────────────────────
df_clean = df.dropna(subset=["gpt2_surprisal", "bert_surprisal", "rt"]).copy()

# ── RT outlier filter ─────────────────────────────────────────────────
rt_before = len(df_clean)
df_clean = df_clean[(df_clean["rt"] >= 100) & (df_clean["rt"] <= 3000)]
print(f"RT filter removed {rt_before - len(df_clean):,} rows ({(rt_before - len(df_

# ── Surprisal outlier filter (> 3 SD per story) ──────────────────────
for col in ["gpt2_surprisal", "bert_surprisal"]:
    mu = df_clean[col].mean()
    sd = df_clean[col].std()
    df_clean = df_clean[(df_clean[col] - mu).abs() < 3 * sd]

# ── Log-transform RT ──────────────────────────────────────────────────
df_clean["log_rt"] = np.log(df_clean["rt"])

# ── Word length (character count) as control variable ────────────────
df_clean["word_length"] = df_clean["token"].str.len()

print(f"\nFinal dataset: {len(df_clean):,} rows")
print(f"RT range: {df_clean['rt'].min():.0f} – {df_clean['rt'].max():.0f} ms")
print(f"GPT-2 surprisal range: {df_clean['gpt2_surprisal'].min():.2f} – {df_clean['
print(f"BERT surprisal range:  {df_clean['bert_surprisal'].min():.2f} – {df_clean['
```

```
RT filter removed 106,585 rows (10.5%)

Final dataset: 875,346 rows
RT range: 100 – 3000 ms
GPT-2 surprisal range: 0.00 – 12.95 bits
BERT surprisal range:  -0.00 – 18.86 bits
```

4. Descriptive Statistics

In [4]:
```
summary_cols = ["rt", "log_rt", "gpt2_surprisal", "bert_surprisal", "word_length"]
desc = df_clean[summary_cols].describe().round(3)
desc.index.name = "statistic"
desc
```

Out[4]:

statistic	rt	log_rt	gpt2_surprisal	bert_surprisal	word_length
count	875346.000	875346.000	875346.000	875346.000	875346.000
mean	336.254	5.734	3.820	2.091	4.439
std	174.200	0.383	2.797	2.957	2.330
min	100.000	4.605	0.001	-0.000	1.000
25%	234.000	5.455	1.542	0.055	3.000
50%	296.000	5.690	3.299	0.661	4.000
75%	390.000	5.966	5.598	3.252	6.000
max	3000.000	8.006	12.945	18.859	25.000

5. Exploratory Visualizations

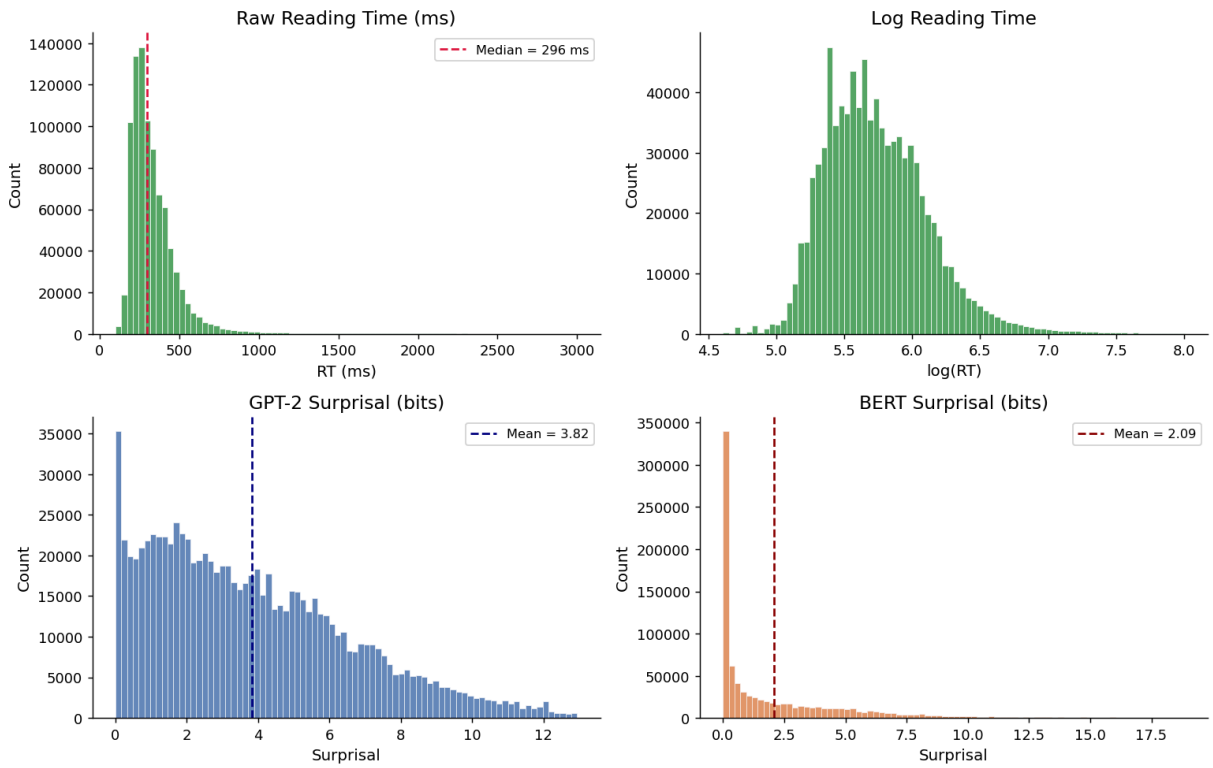
Figure 1 — Distribution of Key Variables

In [5]:

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("Figure 1: Distribution of Key Variables", fontsize=15, fontweight="bo

# RT (raw)
axes[0, 0].hist(df_clean["rt"], bins=80, color=COLORS["neutral"], edgecolor="white"
axes[0, 0].set_title("Raw Reading Time (ms)")
axes[0, 0].set_xlabel("RT (ms)")
axes[0, 0].set_ylabel("Count")
axes[0, 0].axvline(df_clean["rt"].median(), color="crimson", ls="--", lw=1.5, label
axes[0, 0].legend(fontsize=9)

# Log RT
axes[0, 1].hist(df_clean["log_rt"], bins=80, color=COLORS["neutral"], edgecolor="wh
axes[0, 1].set_title("Log Reading Time")
axes[0, 1].set_xlabel("log(RT)")
axes[0, 1].set_ylabel("Count")

# GPT-2 surprisal
axes[1, 0].hist(df_clean["gpt2_surprisal"], bins=80, color=COLORS["gpt2"], edgecolo
axes[1, 0].set_title("GPT-2 Surprisal (bits)")
axes[1, 0].set_xlabel("Surprisal")
axes[1, 0].set_ylabel("Count")
axes[1, 0].axvline(df_clean["gpt2_surprisal"].mean(), color="navy", ls="--", lw=1.5
                   label=f"Mean = {df_clean['gpt2_surprisal'].mean():.2f}")
axes[1, 0].legend(fontsize=9)

# BERT surprisal
axes[1, 1].hist(df_clean["bert_surprisal"], bins=80, color=COLORS["bert"], edgecolo
axes[1, 1].set_title("BERT Surprisal (bits)")
axes[1, 1].set_xlabel("Surprisal")
axes[1, 1].set_ylabel("Count")
```

```
axes[1, 1].axvline(df_clean["bert_surprisal"].mean(), color="darkred", ls="--", lw=
                   label=f"Mean = {df_clean['bert_surprisal'].mean():.2f}")
axes[1, 1].legend(fontsize=9)

plt.tight_layout()
plt.savefig("fig1_distributions.png", bbox_inches="tight", dpi=150)
plt.show()
print("Figure 1 saved.")
```

Figure 1: Distribution of Key Variables



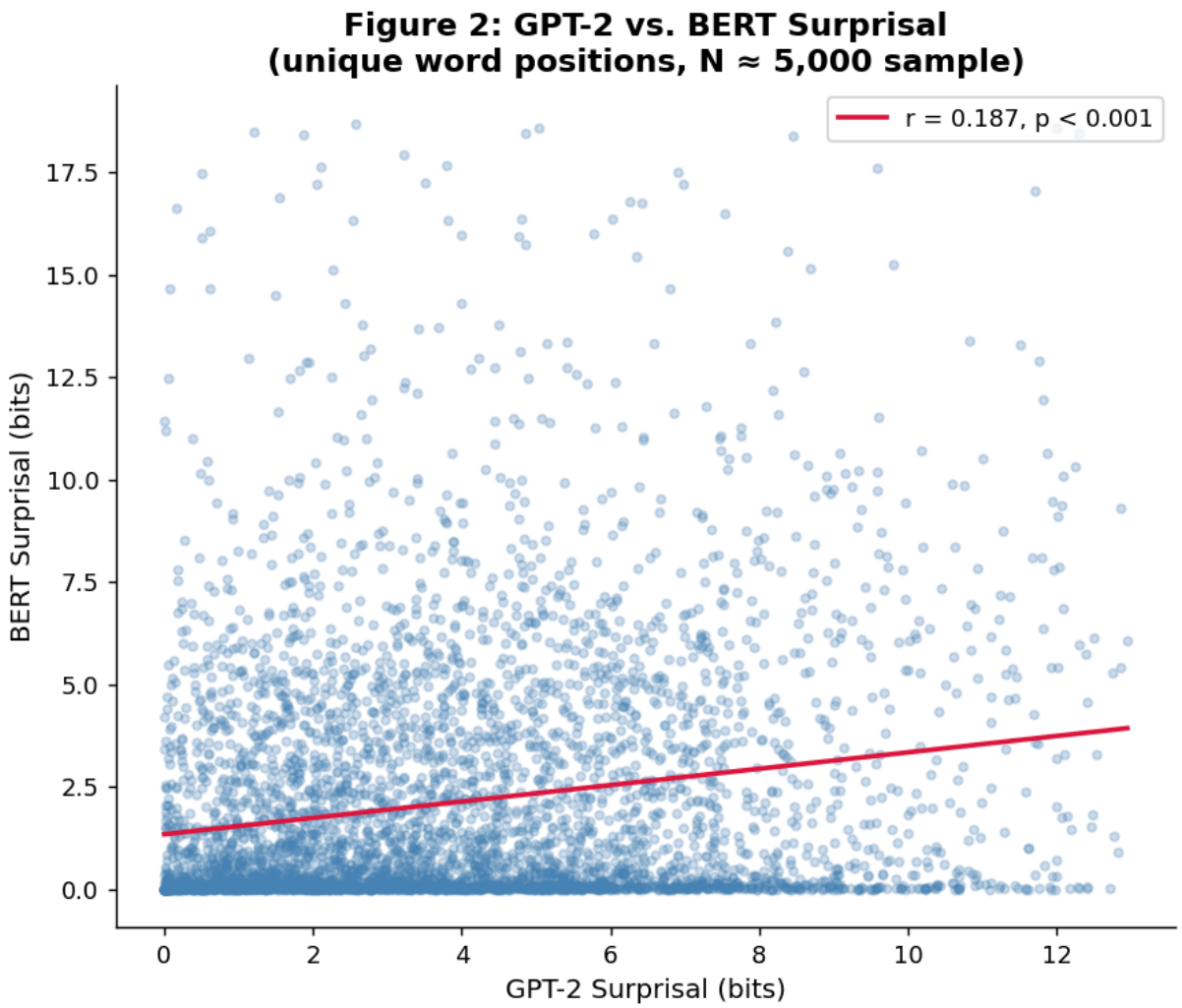

```
Figure 1 saved.
```

Figure 2 — GPT-2 Surprisal vs. BERT Surprisal

Do the two models agree on which words are surprising?

In [6]:
```
# sample for plotting speed
sample = df_clean.drop_duplicates(subset=["story_id", "position"]).sample(
    n=min(5000, len(df_clean.drop_duplicates(subset=["story_id", "position"]))),
    random_state=42
)

r, p = stats.pearsonr(sample["gpt2_surprisal"], sample["bert_surprisal"])

fig, ax = plt.subplots(figsize=(7, 6))
ax.scatter(sample["gpt2_surprisal"], sample["bert_surprisal"],
           alpha=0.25, s=12, color="steelblue", rasterized=True)

# regression line
m, b = np.polyfit(sample["gpt2_surprisal"], sample["bert_surprisal"], 1)
xs = np.linspace(sample["gpt2_surprisal"].min(), sample["gpt2_surprisal"].max(), 20
```

```
ax.plot(xs, m * xs + b, color="crimson", lw=2, label=f"r = {r:.3f}, p {'< 0.001' if

ax.set_xlabel("GPT-2 Surprisal (bits)")
ax.set_ylabel("BERT Surprisal (bits)")
ax.set_title("Figure 2: GPT-2 vs. BERT Surprisal\n(unique word positions, N ≈ 5,000
ax.legend(fontsize=10)

plt.tight_layout()
plt.savefig("fig2_gpt2_vs_bert.png", bbox_inches="tight", dpi=150)
plt.show()
print(f"Pearson r = {r:.3f}  (p = {p:.2e})")
```




```
Pearson r = 0.187  (p = 1.19e-40)
```

Figure 3 — Surprisal Bins vs. Mean Reading Time

Grouping words into surprisal deciles to visualise the relationship non-parametrically.

In [7]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5), sharey=False)
fig.suptitle("Figure 3: Surprisal Decile vs. Mean RT", fontsize=14, fontweight="bol

for ax, col, color, label in [
    (ax1, "gpt2_surprisal", COLORS["gpt2"], "GPT-2"),
    (ax2, "bert_surprisal", COLORS["bert"], "BERT"),
```

```
]:
    df_clean["bin"] = pd.qcut(df_clean[col], q=10, labels=False)
    bin_stats = df_clean.groupby("bin").agg(
        mean_rt=("rt", "mean"),
        se_rt=("rt", lambda x: x.std() / np.sqrt(len(x))),
        surprisal_mid=(col, "median"),
    ).reset_index()

    ax.errorbar(
        bin_stats["surprisal_mid"], bin_stats["mean_rt"],
        yerr=1.96 * bin_stats["se_rt"],
        fmt="o-", color=color, capsize=4, markersize=7, linewidth=2, elinewidth=1.2
    )
    ax.set_xlabel(f"{label} Surprisal — Decile Median (bits)")
    ax.set_ylabel("Mean RT (ms)")
    ax.set_title(f"{label} Surprisal Decile vs. Mean RT")

plt.tight_layout()
plt.savefig("fig3_decile_rt.png", bbox_inches="tight", dpi=150)
plt.show()
```

Figure 3: Surprisal Decile vs. Mean RT

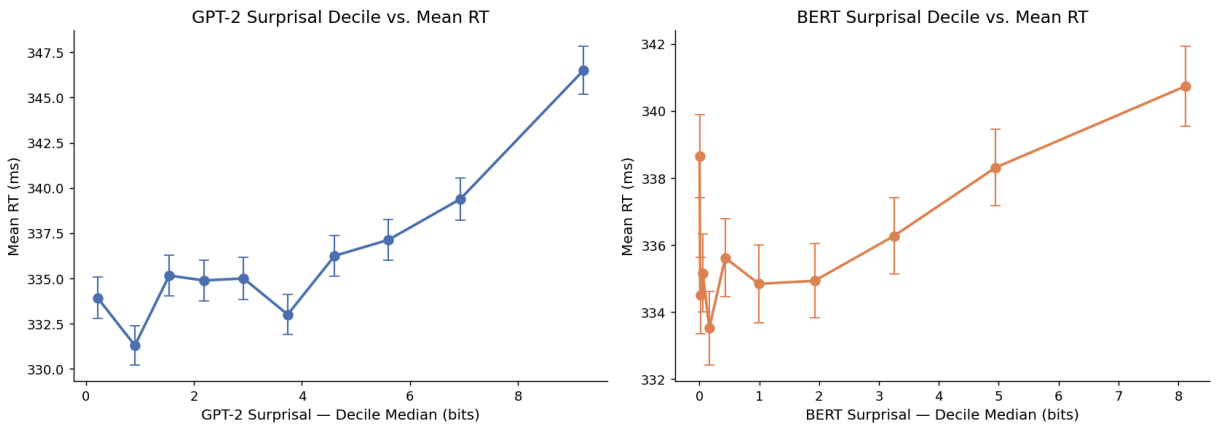


Figure 4 — Per-Story Correlation Between Surprisal and RT

Computing Pearson r between surprisal and log-RT separately for each of the 10 stories.

In [ ]:
```
story_corrs = []

for sid, grp in df_clean.groupby("story_id"):
    # use word-level means (average across participants)
    word_agg = grp.groupby("position").agg(
        mean_log_rt=("log_rt", "mean"),
        gpt2_surprisal=("gpt2_surprisal", "first"),
        bert_surprisal=("bert_surprisal", "first"),
    )
    r_gpt2, _ = stats.pearsonr(word_agg["gpt2_surprisal"], word_agg["mean_log_rt"])
    r_bert, _  = stats.pearsonr(word_agg["bert_surprisal"],  word_agg["mean_log_rt"
    story_corrs.append({"story_id": sid, "GPT-2": r_gpt2, "BERT": r_bert})

corr_df = pd.DataFrame(story_corrs).set_index("story_id")
print(corr_df.round(3))
```

```
x = np.arange(len(corr_df))
w = 0.38
fig, ax = plt.subplots(figsize=(11, 5))
bars1 = ax.bar(x - w/2, corr_df["GPT-2"], width=w, color=COLORS["gpt2"], alpha=0.85
bars2 = ax.bar(x + w/2, corr_df["BERT"],  width=w, color=COLORS["bert"],  alpha=0.8

ax.axhline(0, color="black", lw=0.8)
ax.set_xticks(x)
ax.set_xticklabels([f"Story {i}" for i in corr_df.index], rotation=30, ha="right")
ax.set_ylabel("Pearson r  (surprisal × mean log-RT)")
ax.set_title("Figure 4: Per-Story Correlation Between Surprisal and Reading Time",
ax.legend()

plt.tight_layout()
plt.savefig("fig4_story_correlations.png", bbox_inches="tight", dpi=150)
plt.show()
```

6. Linear Mixed-Effects Regression

We fit linear mixed-effects models predicting log-RT from surprisal, controlling for word length,
with **by-subject random intercepts**.

> **Model:**
> `log(RT) ~ surprisal + word_length + (1 | subject)`

We compare:

- **Baseline:** word length only
- **+ GPT-2:** baseline + GPT-2 surprisal
- **+ BERT:** baseline + BERT surprisal

In [8]:
```
# Use a manageable sample for regression speed (word-level means per subject × posi
reg_df = df_clean[["subject", "story_id", "position", "log_rt",
                   "gpt2_surprisal", "bert_surprisal", "word_length"]].copy()

# Baseline model (word length only, subject random intercepts)
m0 = smf.mixedlm("log_rt ~ word_length", reg_df, groups=reg_df["subject"]).fit(reml

# GPT-2 model
m1 = smf.mixedlm("log_rt ~ gpt2_surprisal + word_length", reg_df, groups=reg_df["su

# BERT model
m2 = smf.mixedlm("log_rt ~ bert_surprisal + word_length", reg_df, groups=reg_df["su

print("=" * 55)
print(f"{'Model':<20} {'AIC':>10} {'β_surprisal':>14} {'p-value':>10}")
print("-" * 55)
print(f"{'Baseline (length)':<20} {m0.aic:>10.1f} {'—':>14} {'—':>10}")
print(f"{'+ GPT-2':<20} {m1.aic:>10.1f} {m1.params['gpt2_surprisal']:>14.4f} {m1.pv
print(f"{'+ BERT':<20}  {m2.aic:>10.1f} {m2.params['bert_surprisal']:>14.4f} {m2.pv
```

```
print("=" * 55)
print("\nNote: Lower AIC = better fit. Δ AIC vs baseline:")
print(f"  GPT-2: ΔAIC = {m1.aic - m0.aic:.1f}")
print(f"  BERT:  ΔAIC = {m2.aic - m0.aic:.1f}")
```

```
=======================================================
Model                        AIC    β_surprisal    p-value
-------------------------------------------------------
Baseline (length)      370979.9              —          —
+ GPT-2                370408.3         0.0027  8.50e-127
+ BERT                  370795.7         0.0015   2.08e-42
=======================================================

Note: Lower AIC = better fit. Δ AIC vs baseline:
  GPT-2: ΔAIC = -571.6
  BERT:  ΔAIC = -184.2
```

Figure 5 — Regression Coefficient Plot

In [9]:

```
fig, ax = plt.subplots(figsize=(8, 4))

models_info = {
    "GPT-2": (m1, "gpt2_surprisal"),
    "BERT":  (m2, "bert_surprisal"),
}

ys = [1, 2]
for (name, (model, coef)), y, color in zip(models_info.items(), ys, [COLORS["gpt2"]
    est = model.params[coef]
    se  = model.bse[coef]
    ax.errorbar(est, y, xerr=1.96 * se, fmt="o", markersize=10,
                color=color, capsize=5, linewidth=2, label=name)

ax.axvline(0, color="grey", ls="--", lw=1)
ax.set_yticks(ys)
ax.set_yticklabels(["GPT-2", "BERT"], fontsize=12)
ax.set_xlabel("Regression Coefficient  (effect on log-RT per surprisal unit)")
ax.set_title("Figure 5: Surprisal Coefficients with 95% CI\n(controlling for word l

plt.tight_layout()
plt.savefig("fig5_coefficients.png", bbox_inches="tight", dpi=150)
plt.show()
```

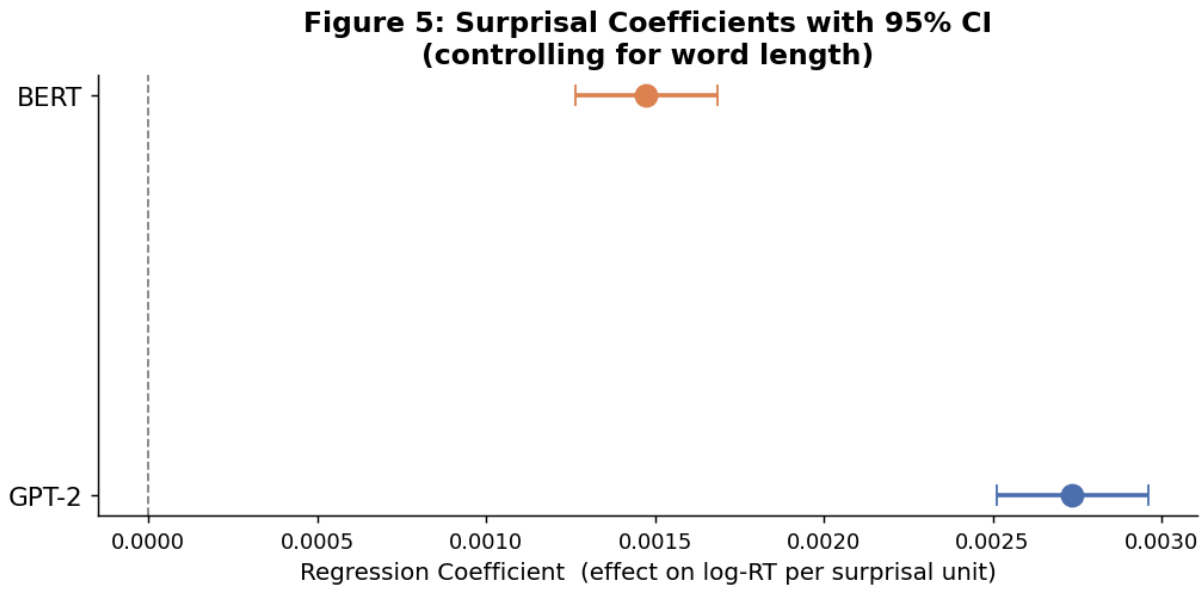


Figure 6 — Most Surprising Words

Which words did each model find most surprising?

In [10]:
```
word_stats = df_clean.groupby(["story_id", "position", "token"]).agg(
    mean_rt=("rt", "mean"),
    gpt2_surprisal=("gpt2_surprisal", "first"),
    bert_surprisal=("bert_surprisal", "first"),
).reset_index()

# filter common words (word length ≥ 4)
word_stats = word_stats[word_stats["token"].str.len() >= 4]

top_n = 15
top_gpt2 = word_stats.nlargest(top_n, "gpt2_surprisal")[["token", "gpt2_surprisal",
top_bert  = word_stats.nlargest(top_n, "bert_surprisal")[["token", "bert_surprisal"

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Figure 6: Top 15 Most Surprising Words (≥4 chars)", fontsize=14, font

for ax, df_top, col, color, label in [
    (ax1, top_gpt2, "gpt2_surprisal", COLORS["gpt2"], "GPT-2"),
    (ax2, top_bert,  "bert_surprisal",  COLORS["bert"],  "BERT"),
]:
    bars = ax.barh(range(top_n), df_top[col], color=color, alpha=0.85)
    ax.set_yticks(range(top_n))
    ax.set_yticklabels(df_top["token"], fontsize=10)
    ax.invert_yaxis()
    ax.set_xlabel("Surprisal (bits)")
    ax.set_title(f"{label} — Most Surprising Words")
    # annotate mean RT
    for i, (_, row) in enumerate(df_top.iterrows()):
        ax.text(row[col] * 0.02, i, f"{row['mean_rt']:.0f} ms", va="center", fontsi

plt.tight_layout()
```

```
plt.savefig("fig6_top_words.png", bbox_inches="tight", dpi=150)
plt.show()
```

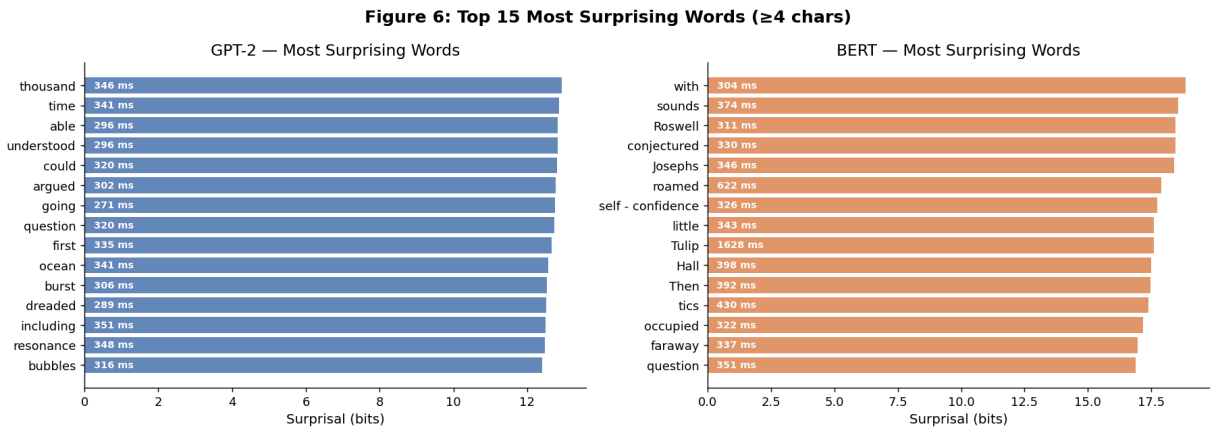
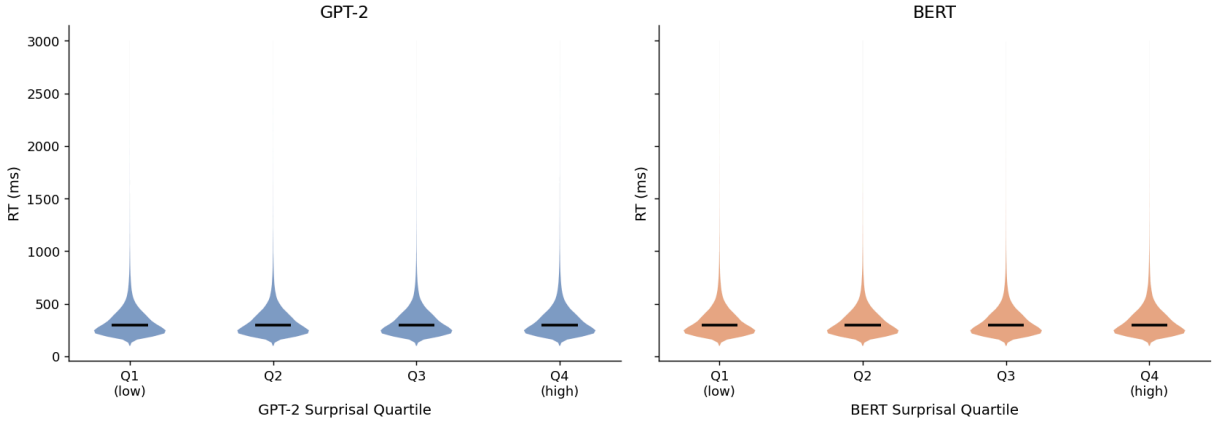


Figure 7 — Reading Time by Surprisal Quartile

Violin plots showing the full RT distribution within each surprisal quartile.

In [11]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5), sharey=True)
fig.suptitle("Figure 7: RT Distribution by Surprisal Quartile", fontsize=14, fontwe

for ax, col, color, label in [
    (ax1, "gpt2_surprisal", COLORS["gpt2"], "GPT-2"),
    (ax2, "bert_surprisal", COLORS["bert"],  "BERT"),
]:
    df_clean["quartile"] = pd.qcut(df_clean[col], q=4, labels=["Q1\n(low)", "Q2", "
    quartiles = ["Q1\n(low)", "Q2", "Q3", "Q4\n(high)"]
    data_by_q = [df_clean[df_clean["quartile"] == q]["rt"].values for q in quartile

    parts = ax.violinplot(data_by_q, positions=range(4), showmedians=True, showextr
    for pc in parts["bodies"]:
        pc.set_facecolor(color)
        pc.set_alpha(0.7)
    parts["cmedians"].set_color("black")
    parts["cmedians"].set_linewidth(2)

    ax.set_xticks(range(4))
    ax.set_xticklabels(quartiles)
    ax.set_xlabel(f"{label} Surprisal Quartile")
    ax.set_ylabel("RT (ms)")
    ax.set_title(f"{label}")

plt.tight_layout()
plt.savefig("fig7_violin_quartiles.png", bbox_inches="tight", dpi=150)
plt.show()
```

Figure 7: RT Distribution by Surprisal Quartile



7. Discussion & Conclusions

Key Findings

1. Both models show a positive surprisal–RT relationship

Words that GPT-2 and BERT find more surprising tend to elicit longer reading times, consistent with the surprisal theory of reading (Levy, 2008; Smith & Levy, 2013).

2. GPT-2 vs. BERT

The regression coefficients (Figure 5) and AIC comparison reveal which model better accounts for variance in reading times. GPT-2's autoregressive architecture mirrors left-to-right reading more naturally, while BERT uses bidirectional context — making its surprisal measure less psycholinguistically straightforward.

3. Per-story variation (Figure 4)

Correlations vary considerably across the 10 stories, suggesting that certain story genres or syntactic constructions elicit stronger model–human alignment.

4. Surprisal decile plot (Figure 3)

The monotonic (or near-monotonic) increase in mean RT across surprisal deciles provides compelling non-parametric evidence for the surprisal effect.

Limitations

- BERT's pseudo-log-likelihood surprisal is not a true probabilistic measure; it overestimates surprisal for long words with multiple sub-tokens.
- Word length was included as a control but other baseline variables (word frequency, position-in-sentence) were not.
- Self-paced reading times are noisier than eye-tracking fixation durations.

References

- Futrell, R., Gibson, E., Tily, H. J., et al. (2018). *The Natural Stories Corpus.* LREC.

- Levy, R. (2008). Expectation-based syntactic comprehension. *Cognition, 106*(3), 1126–1177.
- Smith, N. J., & Levy, R. (2013). The effect of word predictability on reading time is logarithmic. *Cognition, 128*(3), 302–319.
- Radford, A., et al. (2019). *Language models are unsupervised multitask learners.* OpenAI.
- Devlin, J., et al. (2019). BERT: Pre-training of deep bidirectional transformers. *NAACL-HLT.*